

MÓDULO 8: SEGURIDAD

BROKEN ACCESS CONTROL

OWASP TOP 10

BIG school

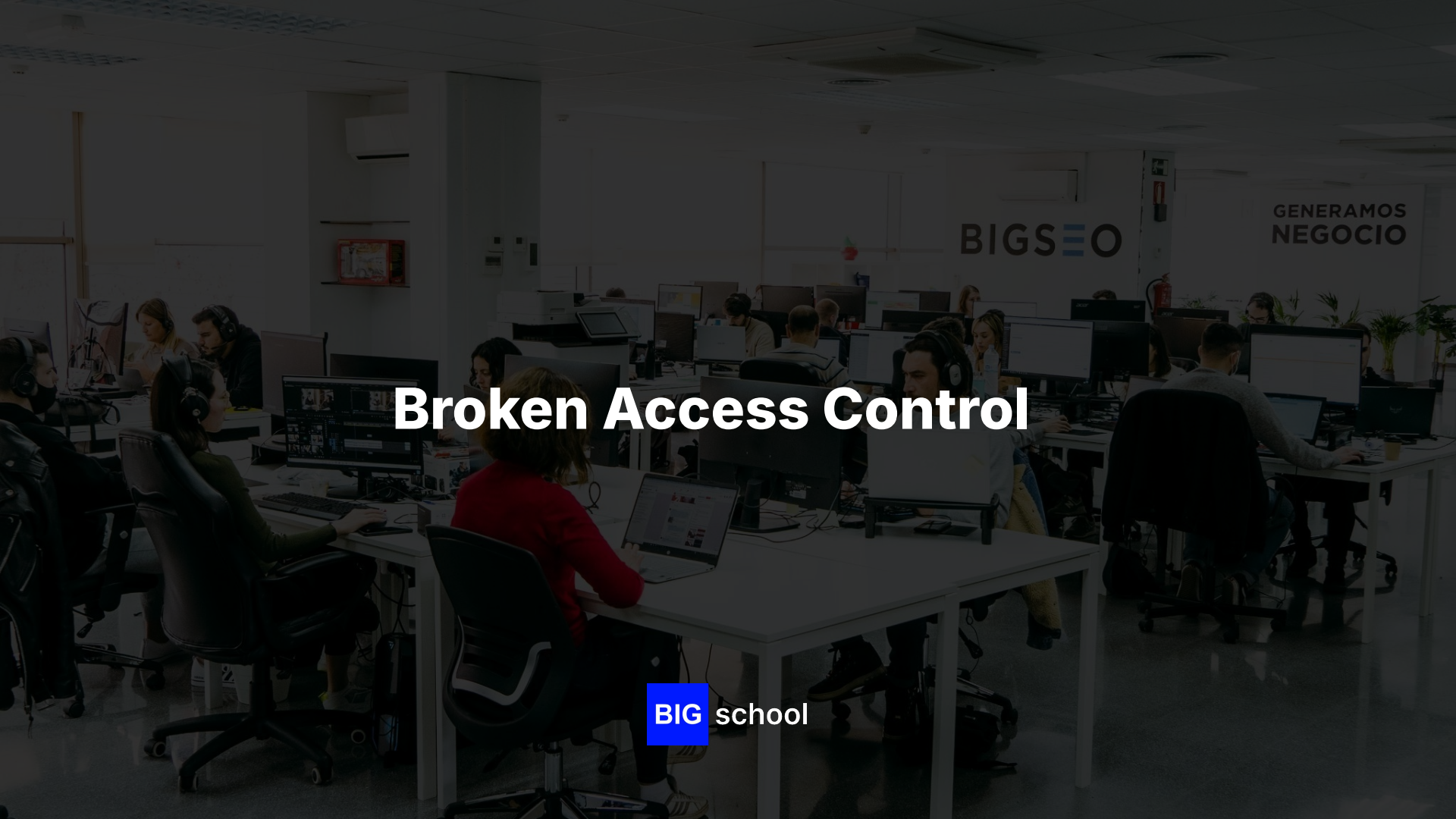
GENERAMOS
NEGOCIO

BIGSEO

Broken Access Control

Índice

- ¿Qué vamos a ver?
- ¿Qué es Broken Access Control?
- ¿En qué consiste?
- Ejemplos típicos
- ¿Qué puede hacer alguien que explote Broken Access Control?
- ¿Cómo detectarlo?
- ¿Cómo prevenirlo?
- ¿Cómo mitigarlo cuando ya está en producción?
- Checklist rápido para desarrolladores
- Conclusiones



Broken Access Control

BIG school

¿Qué vamos a ver?

- Una de las vulnerabilidades más críticas y frecuentes.
- Permite a los usuarios hacer o ver cosas que no deberían.
- Veremos definición, ejemplos, impacto, detección y prevención.

¿Qué es Broken Access

Control?

es cuando no se aplican correctamente permisos sobre recursos.

- Falla la validación de "quién puede hacer qué sobre qué".
- Si la app no valida identidad, rol o recurso, el control de acceso se rompe.

¿En qué consiste?

No es un bug aislado, sino un error de diseño o implementación.

Causas típicas:

- Falta de validación en el servidor.
- Confianza en parámetros de cliente.
- Reglas inconsistentes entre endpoints.
- Roles amplios o sesiones inseguras.
- La autorización debe ser centralizada y siempre en servidor.

Ejemplos típicos

- IDOR: acceso directo a recursos de otros usuarios cambiando un ID.
- Rutas administrativas públicas: endpoints sin control de rol.
- Escalada de privilegios: roles mal definidos que otorgan permisos extra.
- Autenticado $\neq$ autorizado: se valida login pero no el permiso concreto.
- Parámetros del cliente: el servidor confía en datos como "isAdmin=true".

¿Qué puede hacer alguien que explote Broken Access Control?

- Acceso a datos privados o ajenos.
- Modificación no autorizada.
- Escalada de privilegios hasta administrador.
- Ejecución de acciones críticas.
- Compromiso de confidencialidad, integridad y disponibilidad.

¿Cómo detectarlo?

Durante desarrollo:

- Revisiones de código buscando recursos sin control de permisos.
- Pruebas unitarias que validen restricciones por usuario.
- Uso de escáneres automáticos y threat modeling.

En producción:

- Monitoreo de logs y accesos anómalos.
- Alertas por actividad inusual o endpoints sensibles.
- Pentests y auditorías periódicas.

¿Cómo prevenirlo?

Principios de diseño:

- Mínimo privilegio.
- Denegar por defecto.
- Lógica de autorización centralizada.

Buenas prácticas:

- Validar permisos en el servidor.
- No confiar en parámetros del cliente.
- Definir roles y permisos claros (RBAC o ABAC).
- Verificar tokens y sesiones activas.
- Separar endpoints administrativos.

¿Cómo mitigarlo cuando ya está en producción?

- Restringir temporalmente el endpoint afectado.
- Revisar logs y alcance del incidente.
- Rotar credenciales si hubo exposición.
- Aplicar parches y tests regresivos.
- Realizar auditoría y comunicar siguiendo políticas internas.

Checklist rápido para desarrolladores

1. ¿Validas autenticación y autorización en el servidor?
2. ¿Aplicas mínimo privilegio?
3. ¿Evitas usar parámetros del cliente para permisos?
4. ¿Tienes tests de accesos no autorizados?
5. ¿Registras quién accede a qué?
6. ¿Revisas rutas y dependencias expuestas?

Conclusiones

1. Es una falla común pero completamente evitable.
2. Buenas rutinas (middleware, validaciones, tests) previenen incidentes graves.
3. La seguridad de acceso es responsabilidad directa del desarrollador.